

Event Management Backend

Ticketing and Reservations — Project Brief for Interns

Objective: Build a reliable backend service that enables organizations to sell event tickets with a smooth reservation and purchase experience, prevents oversell under concurrency, supports operational workflows (refunds, reconciliation), and provides the business with the metrics and controls needed to run events profitably and safely.

1. Key Stakeholders and Their Needs

Stakeholder	Needs	Success Criteria
Event organizer	Accurate capacity control, simple ticket configuration, attendee lists, refunds, sales reports.	No oversells; timely attendee exports; clear revenue numbers.
End customer (attendee)	Fast reservation flow, clear payment/refund policies, reliable confirmation, timely notifications.	Low checkout failures; predictable hold windows; clear communication.
Operations / Support	Runbook for stuck purchases, reconciliation tools, audit logs, dead-letter queue, refund/reallocation ability.	Quick incident resolution; minimal manual reconciliation.
Finance	Accurate revenue reporting, refund tracking, idempotent order records, exportable transaction history.	Clean month-end reconciliation and audit trail.
Security & Compliance	Rate limits, fraud detection signals, secure handling of payment tokens, data retention policies.	No data breaches; compliant payment handling; documented retention.

2. Business Rules and Policies

Ticket Capacity

- Total capacity is set per ticket type and is **immutable** for reporting.
- Available inventory is the business-facing number that decreases only on **confirmed purchases**.

Reservation Hold Policy

- Hold window default: **10 minutes** (configurable per event).
- Holds do not permanently reduce available inventory. Holds expire automatically and are cleaned up by background jobs.

Purchase Confirmation

- Purchases must be **idempotent**. Clients must supply an **Idempotency-Key**. Duplicate requests with the same key return the same order result.
- Payment capture is **simulated** for the project; design must show where a real payment provider would be integrated.

Refunds and Cancellations

- Refunds must be recorded as separate transactions and increase available inventory **atomically**.
- Partial refunds and partial cancellations must be supported and reflected in audit logs and finance exports.

Overbooking Policy

- **Zero tolerance** for oversell. Any oversell incident triggers immediate alerting and a reconciliation workflow.

Pricing and Fees

- Ticket price stored in **cents**. Support for service fees and tax fields per ticket type.
- Finance expects line-item breakdowns per order for reporting.

Waitlist and Reallocation (Optional)

- If implemented, waitlist entries are **FIFO** and notified when inventory becomes available.

3. User Journeys to Be Documented by Interns

Journey	Flow
Browse and reserve	User selects ticket type and quantity → system creates a short-lived reservation → user receives reservation_id and expires_at.
Complete purchase	User submits payment and Idempotency-Key → system enqueues purchase → consumer finalizes order → user receives confirmation and ticket(s).
Payment failure	Payment fails during confirm → reservation remains until expiry or user cancels → user receives failure notification and retry instructions.
Reservation expiry	Reservation expires → user notified (optional) → inventory remains unchanged.
Refund	Organizer or support issues refund → order marked refunded → inventory incremented and audit recorded → finance export updated.
Manual reconciliation	Ops runs reconciliation script → identifies drift → follows runbook to resolve discrepancies and notify stakeholders.

4. Business Acceptance Criteria

Functional

- Reservations return reservation_id and expires_at.
- Confirmed purchases are idempotent and produce a single order per idempotency key.
- Concurrency test demonstrates no oversell under simulated load.
- Expired reservations cannot be confirmed.
- Refunds update inventory and finance exports correctly.

Operational

- Background expiry job runs on schedule and clears stale holds.
- Dead-letter queue captures permanently failed purchase attempts for manual review.
- Audit trail exists for every inventory change and order lifecycle event.

Performance

- Typical checkout latency target: < 500 ms for reservation creation; < 2 s for enqueueing confirm.
- Queue processing throughput target: configurable; interns must provide expected throughput and a simple load-test script.

Security and Compliance

- Idempotency keys enforced and logged.
- Rate limits applied to reservation and purchase endpoints.
- Sensitive payment data is not stored; only tokens or references are recorded.

5. Business Metrics and KPIs to Deliver

Interns must include how each metric will be measured and a suggested alert threshold for critical metrics.

Category	Metrics
Sales	Orders created per hour, revenue per event, average order value.
Inventory	Remaining inventory per ticket type, active reservations, expired reservations per hour.
Reliability	Idempotency collisions, task failure rate, dead-letter queue size, oversell incidents.
Operational	Queue length per partition, average task processing time, reservation expiry lag.
Customer Experience	Checkout success rate, average time to confirm, refund turnaround time.

6. Reporting and Finance Outputs Required

- Daily sales report per event with: orders, gross revenue, fees, taxes, refunds, net revenue.
- Attendee export CSV with contact info, ticket type, order id, check-in status.
- Audit export for inventory changes with timestamps, actor, and reason.
- Reconciliation report showing $\text{total_capacity} - \text{remaining} - \text{sold} == 0$, or listing discrepancies.

Interns are expected to provide sample CSV schemas and example rows for each export above.

7. Operational Runbook Highlights

Each runbook entry must include the owner role, required commands or dashboards, and expected time-to-resolution.

Incident: Stuck Worker or Backlog

- Steps to inspect worker logs, restart workers, and drain queue safely.

Incident: Oversell Detected

- Immediate steps: pause sales, notify ops and product, run reconciliation, identify offending orders, and follow refund/compensation policy.

Incident: Payment Captured but Order Not Created

- Steps to identify payment token, check DLQ, reconcile, and issue manual order or refund.

Daily Maintenance

- Run reconciliation script, review dead-letter queue, and verify metrics.

8. Rollout and Success Criteria

Pilot Rollout

- Deploy to a staging environment and run a simulated event with seeded users and concurrent checkouts. Validate no oversell and that metrics are emitted.

Go / No-Go Criteria for Production

- Concurrency tests pass; reconciliation shows zero drift; monitoring and alerts configured; runbook validated.

Success After Launch

- No oversell incidents in the first 30 days; checkout success rate > 98%; reconciliation drift = 0.

Prepared as an onboarding reference for the intern team working on the Event Management Backend project.